Oficina Mecânica

Projeto de Banco de Dados – Unidade II – Tema 07

Resumo do que foi feito e roteiro da apresentação

Equipe: Jarlisson Brito Santos, Brunno Passos, Caio Vinicius
Apresentação: 30/11/2026 – 10 minutos por grupo (15h10 às 18h30)
Repositório: `https://github.com/Jarlisson-cpu/oficina-mecanica--bd`

Sumário

Parte 1 – O que foi feito 2

1 O problema 2

2 A solução (arquitetura e modelo) 2

3 Testes de tipos, restrições e regras 2

4 Índices: antes e depois 3

5 Consultas de negócio (C01–C08) 4

6 GitHub e metodologia ágil 4

7 O que foi entregue e onde está 4

8 O que ainda falta 4

Parte 2 – O que vai ser apresentado nos slides 5

8 Roteiro slide a slide 5

9 Dicas para a apresentação 6

10 Perguntas que o professor pode fazer 6

Resumo em uma frase. Montamos um banco de dados em PostgreSQL que impede os erros mais comuns de uma oficina mecânica (placa duplicada, peça usada sem baixa no estoque, OS sem responsável, hodômetro voltando e OS fechada sendo alterada) e provamos cada regra com teste: **34 de 34 testes passaram** e os índices deixaram a busca por placa **cerca de 120 vezes mais rápida** (30,60 ms $\rightarrow$ 0,25 ms).

Parte 1 – O que foi feito

1. O problema

Na oficina, o controle em papel ou planilha gera falhas recorrentes. Cada uma virou uma regra no banco e um teste:

Problema	Como o banco resolve	Teste que prova
Placa ou CPF cadastrado duas vezes	`UNIQUE` + domínios de formato	R02, R03, T02, T10
Peça usada sem baixa no estoque	trigger de baixa/estorno + `CHECK (qtd_estoque >= 0)`	G01, G02, G08
OS sem mecânico responsável	`NOT NULL` + chave estrangeira + trigger "mecânico ativo"	G05
Hodômetro "voltando"	trigger compara com o km já registrado	G06
OS fechada sendo alterada	`CHECK`s de data/status + trigger de ciclo de vida	R07–R09, G04, G07
Histórico por placa demorado	índice `(id_veiculo, data_abertura DESC)`	B1: 30,60 → 0,25 ms

2. A solução (arquitetura e modelo)

- **SGBD:** PostgreSQL 16, schema `oficina`.
- **6 tabelas:** `cliente`, `veiculo`, `mecanico`, `ordem_servico`, `peca` e `os_peca` (associativa N:N entre OS e peça, com chave primária composta).
- **Domínios** (regra de formato escrita uma vez só): placa antiga e Mercosul, CPF/CNPJ só com dígitos, telefone, dinheiro e status como `ENUM`.
- **5 regras de negócio em triggers:** copiar o preço da peça; bloquear alteração de OS fechada; baixa/estorno de estoque; mecânico ativo e hodômetro; ciclo de vida da OS e cálculo da mão de obra (horas × valor-hora).
- **View** `vw_os_total`: valor de cada OS (peças + mão de obra − desconto).
- **8 índices**, cada um ligado a uma consulta real da oficina.
- **Dados de teste:** 20 mil clientes, 30 mil veículos, 300 mil OS e 450 mil itens de peça (fictícios, gerados por script).

3. Testes de tipos, restrições e regras

- **34 testes automatizados**: 11 de tipos (T), 13 de restrições (R) e 10 de regras/triggers (G). **Todos passaram.**
- Nos 26 testes em que o banco deveria recusar o comando, ele devolveu **exatamente o código de erro esperado** (ex.: 23505 = valor duplicado, 23503 = chave estrangeira, 23514 = CHECK).
- Cada teste roda dentro de `BEGIN ... ROLLBACK` (não suja o banco) e gera um log `.txt` e uma captura de tela `.png`.

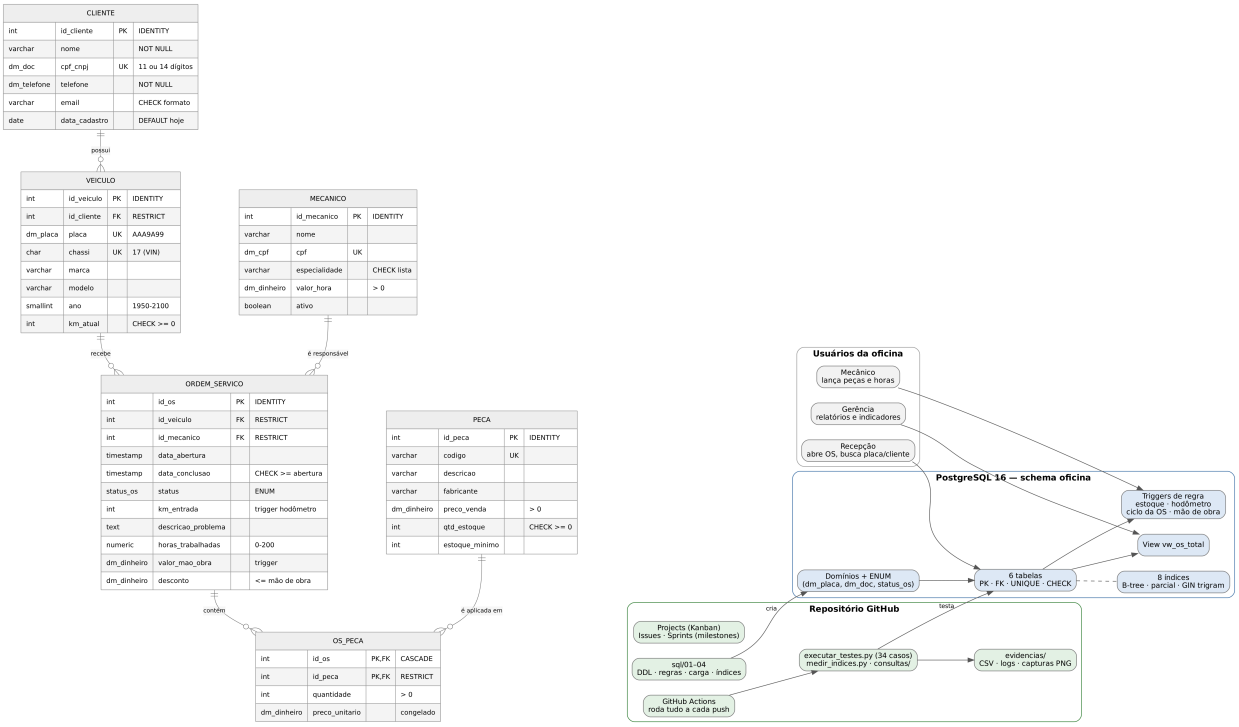


Figura 1: Diagrama ER (esquerda) e arquitetura da solução (direita).

- Destaques para mostrar: **T07** (`FLOAT` erra $0{,}1+0{,}2$ e `NUMERIC` não), **G02** (estoque insuficiente barrado) e **G10** (fluxo completo: abrir, lançar peças, concluir e faturar R$ 348,40).

4. Índices: antes e depois

Medido com `EXPLAIN ANALYZE`, mediana de 7 execuções:

Consulta	Antes	Depois	Páginas lidas
B1 Histórico por placa	30,60 ms	0,25 ms	8.323 → 31
B2 Painel de OS pendentes (índice parcial)	24,80 ms	0,35 ms	8.276 → 52
B3 Produção do mecânico no mês	24,32 ms	0,84 ms	8.204 → 200
B4 Faturamento do mês	23,52 ms	6,28 ms	8.204 → 2.590
B5 Consumo de uma peça	29,25 ms	2,56 ms	2.870 → 611
B6 Veículos de um cliente	2,00 ms	0,14 ms	613 → 4
B7 Cliente por parte do nome (GIN)	12,04 ms	0,97 ms	266 → 174
B8 OS concluídas (95% da tabela)	46,32 ms	49,09 ms	**índice ignorado**
B9 OS abertas (0,2% da tabela)	25,46 ms	1,59 ms	8.204 → 532
W1 Inserir 20 mil OS (custo de escrita)	1.250 ms	1.578 ms	+26% mais lento

O que aprendemos (resultados “contra a intuição”):

- Índice em coluna com poucos valores diferentes (`status`) é **ignorado** quando o filtro pega quase a tabela toda (B8), mas é usado quando pega pouco (B9).

- Índice tem custo: inserir ficou **26% mais lento** e os índices ocupam cerca de 40 MB.
- A view com `LATERAL` ficou **633× mais rápida** para uma placa, mas **2× mais lenta** no relatório geral; por isso o relatório geral soma direto nas tabelas.
- Os testes acharam um **erro numa consulta de estoque** (C04 somava todos os períodos); ele foi corrigido.

5. Consultas de negócio (C01–C08)

C01 histórico por placa; C02 faturamento mensal (tíquete médio entre R$ 748 e R$ 770); C03 ranking de mecânicos (`RANK()`); C04 peças para repor; C05 OS atrasadas; C06 top 10 clientes; C07 taxa de retrabalho por mecânico (`LAG()`, maior taxa 1,85%); C08 conferência: total da view = total das tabelas (R$ 216.799.627,25 nos dois).

6. GitHub e metodologia ágil

- **Repositório público:** `https://github.com/Jarlisson-cpu/oficina-mecanica--bd`
- **Scrum simplificado** com 5 sprints semanais (*Milestones*), até 30/11.
- **18 tarefas** (*Issues*) com rótulos: 13 concluídas e 5 abertas.
- **GitHub Actions:** a cada envio o GitHub recria o banco e roda os 34 testes sozinho (selo verde no README).
- **Definição de pronto:** SQL no repositório + teste automatizado + evidência gerada.

7. O que foi entregue e onde está

Pasta `Documentos\Projeto BD - Oficina Mecanica`:

Pasta	Conteúdo
00 - Enunciado e Modelos	enunciado do professor e modelos SBC
01 - Artigo	artigo em PDF, Word (.docx) e LaTeX (.tex) + figuras
02 - Banco de Dados	scripts SQL 01 a 04 e as consultas C01–C08
03 - Testes e Evidencias	resumo dos testes, tabelas CSV, capturas, logs e planos de execução
04 - Diagramas	diagrama ER e arquitetura
05 - Repositorio GitHub	cópia compactada do repositório
Resumo e Roteiro	este documento

8. O que ainda falta

- Preencher no artigo: **[Nome do(a) Professor(a)]** e **[Curso]**.
- Criar o **quadro Kanban** no GitHub (aba Projects) e adicionar `caiosouza0121` e `devebruzzz` como colaboradores (Settings → Collaborators).
- Montar os **slides** (roteiro na Parte 2) e **ensaiar** com cronômetro.

Parte 2 – O que vai ser apresentado nos slides

Tempo total: **10 minutos**, com 10 slides. A divisão por pessoa é uma sugestão; ajustem como preferirem. Regra de ouro do professor: *toda afirmação vem com um teste*, então cada slide de resultado mostra uma captura ou tabela real.

8. Roteiro slide a slide

Slide 1 – Capa 0:00–0:20 • Jarlisson

Título do projeto, tema 07, nomes do grupo, professor, data.
Fala: "Nosso tema é a oficina mecânica: controle de ordens de serviço, peças e mecânicos."

Slide 2 – O problema 0:20–1:20 • Jarlisson

Lista das 6 falhas do controle em papel/planilha (tabela da seção 1, sem a coluna de testes).
Fala: "As regras existem na cabeça das pessoas, mas nada obriga que sejam cumpridas. A ideia foi colocar essas regras dentro do banco."

Slide 3 – Arquitetura da solução 1:20–2:20 • Jarlisson

Figura da arquitetura: usuários → PostgreSQL (domínios, tabelas, triggers, view, índices) ← GitHub (scripts, testes, Actions).
Fala: explicar as 3 camadas em 1 minuto.

Slide 4 – Modelo de dados (DER) 2:20–3:20 • Brunno

Diagrama ER com as 6 tabelas.
Destaques: `os_peca` resolve o N:N com chave composta; o preço da peça fica "congelado" na OS; `RESTRICT` e `CASCADE` nas chaves estrangeiras.

Slide 5 – Decisões de tipos 3:20–4:10 • Brunno

Tabela tipo → justificativa → teste. Captura do **T07**: `FLOAT` dá 0,30000000000000004 e `NUMERIC` dá 0,3.
Fala: "Dinheiro nunca em FLOAT – e o teste prova."

Slide 6 – Restrições e regras (triggers) 4:10–5:30 • Brunno

Lista das 5 regras + captura do **G02** (estoque insuficiente bloqueado com erro 23514).
Fala: "O trigger só subtrai; quem barra o estoque negativo é o CHECK. Assim a regra vale até para atualização manual."

Slide 7 – Resultado dos 34 testes 5:30–6:30 • Caio

Resumo: 34/34 passaram; 26 erros esperados com o código exato. Mostrar a captura do **G10** (fluxo completo, R$ 348,40).
Opcional ao vivo (30 s): abrir o GitHub e mostrar o selo verde do Actions.

Slide 8 – Índices: antes × depois 6:30–8:00 • Caio

Tabela B1–B9 + W1 e as capturas do plano B1 (*Seq Scan* → *Index Scan*).
Fala: “Histórico por placa: de 30,6 para 0,25 ms, lendo 31 páginas em vez de 8.323.”

Slide 9 – O que não funcionou como esperávamos 8:00–9:00 • Caio

B8 (índice ignorado com 95% da tabela), +26% na escrita, view `LATERAL` 633× mais rápida num caso e 2× mais lenta noutro, erro achado na C04.
Fala: “Nenhum desses resultados teria aparecido sem teste.”

Slide 10 – Metodologia ágil e conclusão 9:00–10:00 • Jarlisson

Print do GitHub: Milestones (5 sprints), Issues e quadro Kanban. Conclusão em 3 tópicos: as regras estão no banco; tudo foi testado; índice só com medição.
Fechamento: link do repositório e “Obrigado, perguntas?”

9. Dicas para a apresentação

- **Ensaiar com cronômetro** pelo menos duas vezes: 10 minutos passam rápido.
- Usar as **capturas reais** da pasta `03 - Testes e Evidencias\Capturas de tela`.
- Deixar o GitHub **aberto numa aba** (README com o selo verde e a aba Issues).
- Ter o **PDF do artigo** aberto, caso o professor peça um detalhe.

10. Perguntas que o professor pode fazer

Pergunta	Resposta curta
Por que `NUMERIC` e não `FLOAT` para dinheiro?	`FLOAT` é binário e acumula erro; o T07 mostra $0{,}1 + 0{,}2 \neq 0{,}3$.
Por que trigger e não só `CHECK`?	`CHECK` só enxerga a própria linha; baixa de estoque e km do veículo dependem de outra tabela.
Por que o índice em `status` não foi usado?	Com 95% das linhas, ler a tabela inteira em sequência é mais barato que ir e voltar entre índice e tabela.
Índice parcial, o que é?	Índice só das linhas que atendem um filtro (OS pendentes): 464 kB em vez de cerca de 7 MB.
Os dados são reais?	Não; são fictícios e gerados por script, com 300 mil OS para os índices fazerem diferença mensurável.
Como garantem que o teste passou pelo motivo certo?	Comparamos o código SQLSTATE do erro, não só se deu erro.
O que acontece se dois mecânicos usarem a mesma peça ao mesmo tempo?	Não testamos concorrência; está nas limitações e nos trabalhos futuros.